

ขั้นตอนการเขียน next js + mysql + orm prisma + axios

1. สร้างฐานข้อมูล bookstore_db



2. ติดตั้ง npm install -D prisma@

- npm install @prisma/client@6

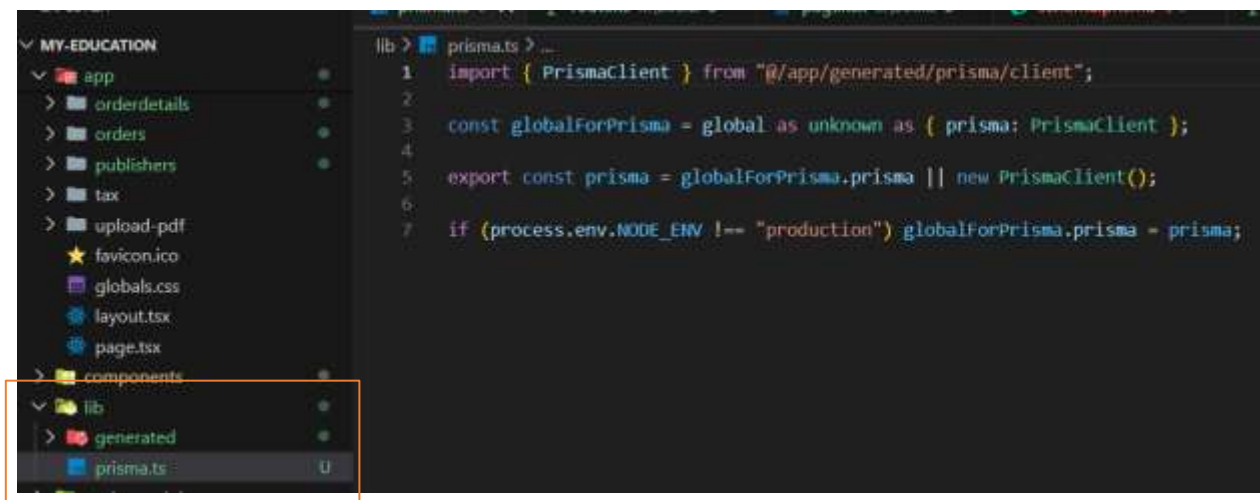
3. นำเข้า prisma

- npx prisma init

4. generate

- npx prisma generate

- สร้างไฟล์ชื่อ prisma.ts ในโฟลเดอร์ lib



lib/prisma.ts

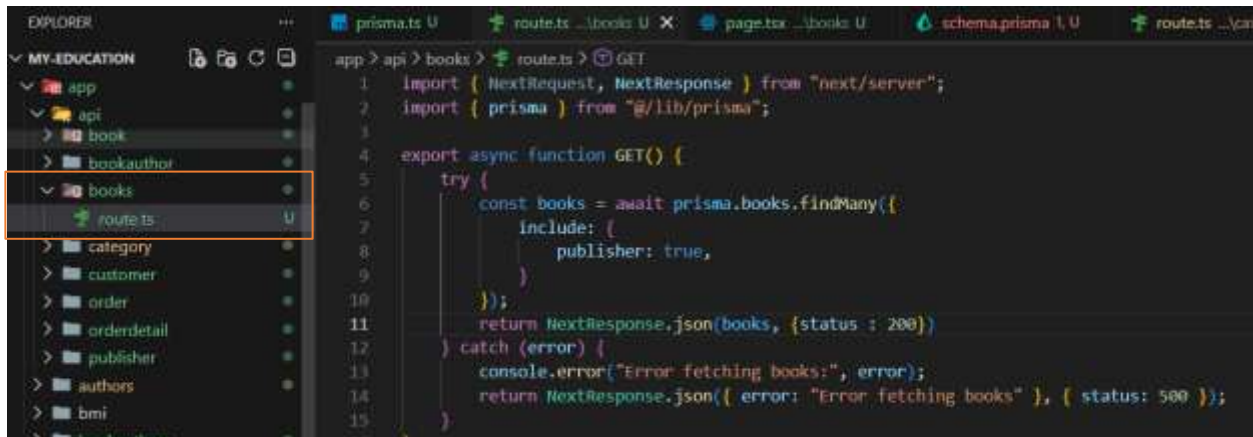
```
import { PrismaClient } from "@generated/prisma/client";

const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

- สร้างไฟล์ route.ts ในโฟลเดอร์ books



/api/books/route.ts

```
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";

export async function GET() {
  try {
    const books = await prisma.books.findMany({
      include: {
        publisher: true,
      }
    });

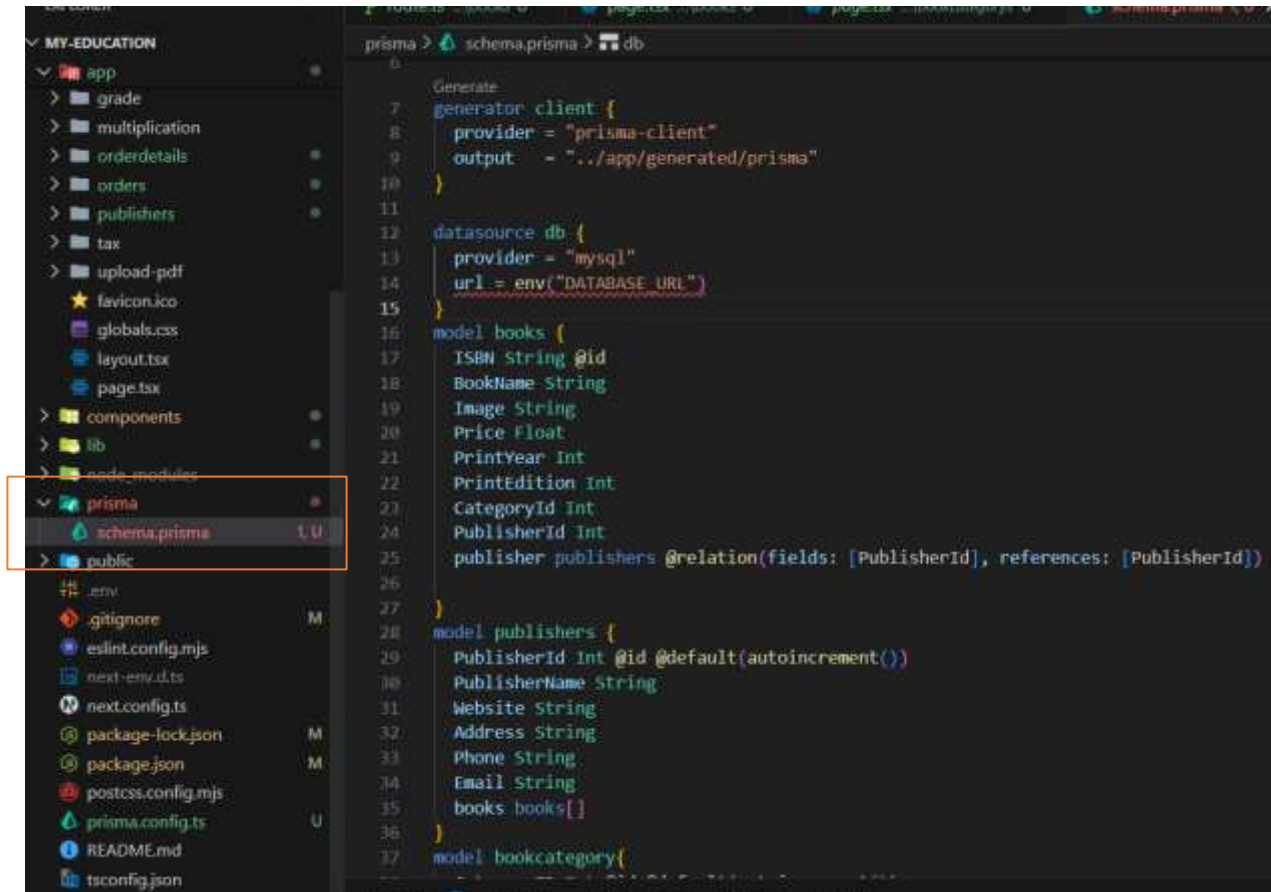
    return NextResponse.json(books, {status : 200})
  } catch (error) {
    console.error("Error fetching books:", error);
    return NextResponse.json({ error: "Error fetching books" }, { status: 500 });
  }
}
```

```

    } catch (error) {
      console.error("Error fetching books:", error);
      return NextResponse.json({ error: "Error fetching books" }, { status: 500 });
    }
  }
}

```

- สร้างไฟล์ schema.prisma ในโฟลเดอร์ prisma



prisma/ schema.prisma

// This is your Prisma schema file,

// learn more about it in the docs: <https://pris.ly/d/prisma-schema>

// Looking for ways to speed up your queries, or scale easily with your serverless or edge functions?

```
// Try Prisma Accelerate: https://pris.ly/cli/accelerate-init
```

```
generator client {  
  provider = "prisma-client"  
  output   = "../generated/prisma"  
}
```

```
datasource db {  
  provider = "mysql"  
  url      = env("DATABASE_URL")  
}
```

```
model books{  
  ISBN String @id  
  BookName String  
  Price Float  
  Image String  
  PrintYear Int  
  PrintEdition Int  
  CategoryID Int  
  PublisherID Int  
  publisher publishers @relation(fields: [PublisherID], references: [PublisherID])  
}
```

```
model publishers{  
  PublisherID Int @id @default(autoincrement())  
  PublisherName String
```

Website String

Address String

Phone String

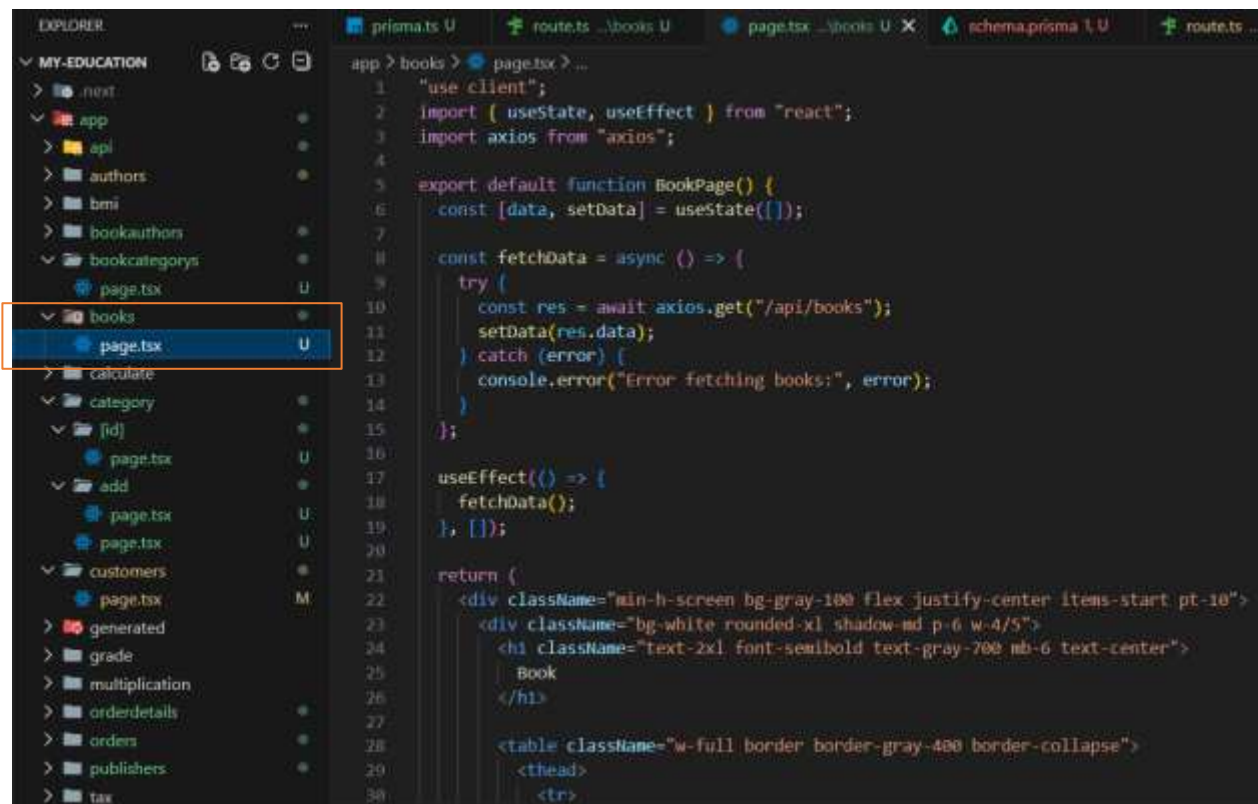
Email String

books books[]

}

npm i axios

- app/books/page.tsx



```
1 "use client";
2 import { useState, useEffect } from "react";
3 import axios from "axios";
4
5 export default function BookPage() {
6   const [data, setData] = useState([]);
7
8   const fetchData = async () => {
9     try {
10       const res = await axios.get("/api/books");
11       setData(res.data);
12     } catch (error) {
13       console.error("Error fetching books:", error);
14     }
15   };
16
17   useEffect(() => {
18     fetchData();
19   }, []);
20
21   return (
22     <div className="min-h-screen bg-gray-100 flex justify-center items-start pt-10">
23       <div className="bg-white rounded-xl shadow-md p-6 w-4/5">
24         <h1 className="text-2xl font-semibold text-gray-700 mb-6 text-center">
25           Book
26         </h1>
27
28         <table className="w-full border border-gray-400 border-collapse">
29           <thead>
30             <tr>
```

books/page.tsx

"use client";

import { useState, useEffect } from "react";

import axios from "axios";

```
export default function BookPage() {
  const [data, setData] = useState([]);

  const fetchData = async () => {
    try {
      const res = await axios.get("/api/books");
      setData(res.data);
    } catch (error) {
      console.error("Error fetching books:", error);
    }
  };

  useEffect(() => {
    fetchData();
  }, []);

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center items-start pt-10">
      <div className="bg-white rounded-xl shadow-md p-6 w-4/5">
        <h1 className="text-2xl font-semibold text-gray-700 mb-6 text-center">
          Book
        </h1>

        <table className="w-full border border-gray-400 border-collapse">
          <thead>
            <tr>
```

```

<th className="border border-gray-400 px-3 py-2 text-left">
  รหัส
</th>
<th className="border border-gray-400 px-3 py-2 text-left">
  ชื่อหนังสือ
</th>
<th className="border border-gray-400 px-3 py-2 text-left">
  ราคา
</th>
<th className="border border-gray-400 px-3 py-2 text-left">
  สำนักพิมพ์
</th>
<th className="border border-gray-400 px-3 py-2 text-left">
  ปีที่ตีพิมพ์
</th>
</tr>
</thead>

<tbody>
{data.map((book: any) => (
  <tr key={book.ISBN}>
    <td className="border border-gray-400 px-3 py-2">
      {book?.ISBN}
    </td>
    <td className="border border-gray-400 px-3 py-2">
      {book?.BookName}
    </td>

```

```

        <td className="border border-gray-400 px-3 py-2">
            {book?.Price}
        </td>

        <td className="border border-gray-400 px-3 py-2">
            {book?.publisher?.PublisherName}
        </td>

        <td className="border border-gray-400 px-3 py-2">
            {book?.PrintYear}
        </td>
    </tr>
    )}
</tbody>
</table>
</div>
</div>
);
}

```

ผลลัพธ์

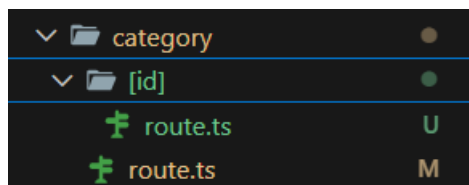
Book				
รหัส	ชื่อหนังสือ	ราคา	สำนักพิมพ์	ปีที่ตีพิมพ์
978001	เขียน SQL สำหรับมือใหม่	299	สำนักพิมพ์ ABC	2024
978002	นิยายรักลึกลับ	199	สำนักพิมพ์ DEF	2023

เพิ่มลบแก้ไข category

1. เพิ่ม code ใน prisma

```
}  
model bookcategory{  
  CategoryID Int @id @default(autoincrement())  
  CategoryName String  
}
```

2. สร้างไฟล์เตอร์ชื่อ category/route.ts และสร้าง /id/router.ts



3. สร้าง POST API (Insert ข้อมูล)

- การ GET API

```
export async function GET(request: NextRequest) {  
  try {  
    const categories = await prisma.bookcategory.findMany();  
    return NextResponse.json(categories, {status: 200});  
  } catch (error) {  
    console.error("Error fetching category:", error);  
    return NextResponse.json({ error: "Failed to fetch category" }, { status: 500 });  
  }  
}
```

- การ POST API

```

export async function POST(request: NextRequest) {
  try {
    const { CategoryName } = await request.json();
    const newCategory = await prisma.bookcategory.create({
      data: {
        CategoryName,
      }
    });
    return NextResponse.json(newCategory, { status: 201 });
  } catch (error) {
    console.error("Error creating category:", error);
    return NextResponse.json({ error: "Failed to create category" }, { status: 500 });
  }
}

```

- การ PUT API

```

export async function PUT(request: NextRequest) {
  try {
    const { CategoryName } = await request.json();
    const { searchParams } = new URL(request.url);
    const id = searchParams.get("id");

    const updatedCategory = await prisma.bookcategory.update({
      where: { CategoryID: parseInt(id!) },
      data: {
        CategoryName,
      }
    });

    return NextResponse.json(updatedCategory, { status: 200 });
  } catch (error) {
    console.error("Error updating category:", error);
    return NextResponse.json({ error: "Failed to update category" }, { status: 500 });
  }
}

```

- การ DELETE API

```
export async function DELETE(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url); // api/category?id=1
    const id = searchParams.get("id"); //1
    await prisma.bookcategory.delete({
      where: { CategoryID: parseInt(id!) }, // DELETE FROM book_categories WHERE category_id = 1
    });
    return NextResponse.json({ message: "Category deleted successfully" }, { status: 200 });
  } catch (error) {
    console.error("Error deleting category:", error);
    return NextResponse.json({ error: "Failed to delete category" }, { status: 500 });
  }
}
```

category/route.ts

```
import { NextResponse, NextRequest } from "next/server";
```

```
import { prisma } from "@lib/prisma";
```

```
import { count } from "console";
```

```
export async function GET(request: NextRequest) {
```

```
  try {
```

```
    const categories = await prisma.bookcategory.findMany();
```

```
    return NextResponse.json(categories, {status: 200});
```

```
  } catch (error) {
```

```
    console.error("Error fetching category:", error);
```

```
    return NextResponse.json({ error: "Failed to fetch category" }, { status: 500 });
```

```
  }
```

```
}
```

```
export async function POST(request: NextRequest) {
```

```
  try {
```

```
    const { CategoryName } = await request.json();
```

```
    const newCategory = await prisma.bookcategory.create({
```

```

        data: {
            CategoryName,
        }
    });
    return NextResponse.json(newCategory, { status: 201 });
} catch (error) {
    console.error("Error creating category:", error);
    return NextResponse.json({ error: "Failed to create category" }, { status: 500 });
}
}

```

```

export async function PUT(request: NextRequest) {
    try {
        const { CategoryName } = await request.json();
        const { searchParams } = new URL(request.url);
        const id = searchParams.get("id");

        const updatedCategory = await prisma.bookcategory.update({
            where: { CategoryID: parseInt(id!) },
            data: {
                CategoryName,
            }
        });

        return NextResponse.json(updatedCategory, { status: 200 });
    } catch (error) {
        console.error("Error updating category:", error);
    }
}

```

```

    return NextResponse.json({ error: "Failed to update category" }, { status: 500 });
  }
}

export async function DELETE(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url); // api/category?id=1
    const id = searchParams.get("id"); //1
    await prisma.bookcategory.delete({
      where: { CategoryID: parseInt(id!) }, // DELETE FROM book_categories
      WHERE category_id = 1
    });
    return NextResponse.json({ message: "Category deleted successfully" }, {
status: 200 });
  } catch (error) {
    console.error("Error deleting category:", error);
    return NextResponse.json({ error: "Failed to delete category" }, { status: 500 });
  }
}

```

category/id/route.ts

```

import { NextResponse, NextRequest } from "next/server";
import { prisma } from "@/lib/prisma";

export async function GET(
  request: NextRequest,

```

```

context: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await context.params;

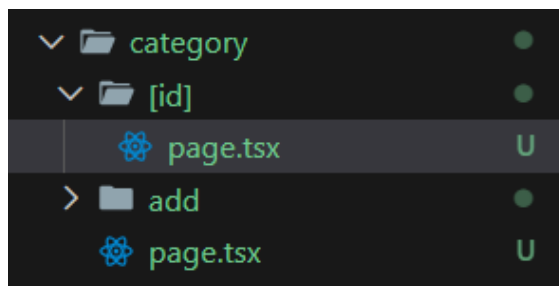
    const categories = await prisma.bookcategory.findUnique({
      where: {
        CategoryID: Number(id)
      }
    });

    return NextResponse.json(categories);

  } catch (error) {
    console.error("Error fetching categories:", error);
    return NextResponse.json(
      { error: "Failed to fetch categories" },
      { status: 500 }
    );
  }
}

```

4. สร้างฟอร์มแก้ไขข้อมูล category/id/page.tsx



category/id/page.tsx

```
"use client";

import { useState, useEffect } from "react";
import axios from "axios";
import { useParams } from "next/navigation"; // /category/7

export default function page() {
  const [category, setCategory] = useState({ CategoryName: "" });
  const [status, setStatus] = useState("");
  const { id } = useParams();

  useEffect(() => {
    if (!id) return;

    const fetchCategory = async () => {
      try {
        const response = await axios.get(`/api/category/${id}`);
        setCategory(response.data);
      } catch (error) {
        console.error("Error fetching categories:", error);
      }
    };

    fetchCategory();
  }, [id]);

  console.log(category?.CategoryName);
}
```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
    await axios.put(`/api/category/?id=${id}`, {
      CategoryName: category?.CategoryName,
    });
    setCategory({ CategoryName: "" });
    setStatus("บันทึกข้อมูลสำเร็จ!");
  } catch (error) {
    console.error("Error adding category:", error);
  }
};

return (
  <div>
    <h1 className="text-2xl font-bold mb-4">เพิ่มข้อมูลหมวดหมู่</h1>
    {status && <p className="mb-4 text-green-500">{status}</p>}
    <form onSubmit={handleSubmit} className="max-w-sm">
      <div className="mb-4">
        <label
          htmlFor="categoryName"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          ชื่อหมวดหมู่
        </label>
        <input
          type="text"

```



```

        id="categoryName"
        value={category.CategoryName || ""}
        onChange={(e) => setCategory({ CategoryName: e.target.value })}
        className="w-full border border-gray-300 rounded px-3 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500"
      />
    </div>
    <button
      type="submit"
      className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600
transition-colors duration-300"
      >
      แก้ไขหมวดหมู่
    </button>
  </form>
</div>
);
}

```

- สร้างฟอร์มเพิ่มข้อมูล category/add/page.tsx

```

"use client";
import { useState, useEffect } from "react";
import axios from "axios";

export default function page() {
  const [category, setCategory] = useState([]);

```

```
useEffect(() => {
  const fetchCategory = async () => {
    try {
      const response = await axios.get("/api/category");
      setCategory(response.data);
    } catch (error) {
      console.error("Error fetching category:", error);
    }
  };
  fetchCategory();
}, []);

const handleDelete = async (id: number) => {
  try {
    await axios.delete(`/api/category?id=${id}`);
    setCategory(category.filter((cat: any) => cat.category_id !== id));
  } catch (error) {
    console.error("Error deleting category:", error);
  }
};

return (
  <div>
    <a
      href="/category/add"
      className="text-2xl font-bold mb-4 text-blue underline"
    >
```

เพิ่มข้อมูลหมวดหมู่

<table className="border p-3">

<thead>

<tr>

<th className="border p-2">Category ID</th>

<th className="border p-2">Category Name</th>

<th className="border p-2">Actions</th>

</tr>

</thead>

<tbody>

{category.map((cat: any) => (

<tr key={cat.CategoryID}>

<td className="border p-2">{cat.CategoryID}</td>

<td className="border p-2">{cat.CategoryName}</td>

<td className="border p-2">

<a

href={` /category/\${cat.CategoryID}`}

className="bg-blue-500 text-white px-2 py-1 rounded me-1"

>

Edit

<button

onClick={() => handleDelete(cat.CategoryID)}

className="bg-red-500 text-white px-2 py-1 rounded"

>

Delte

```

        </button>
    </td>
</tr>
)}}
</tbody>
</table>
</div>
);
}

```

ผลลัพธ์

Category		
+ เพิ่มข้อมูลหมวดหมู่		
Category ID	Category Name	Actions
1	นิยาย	Edit Delete
2	การศึกษา	Edit Delete
3	คอมพิวเตอร์	Edit Delete
4	บ้านและสวน	Edit Delete
5	การดูแลสุขภาพ	Edit Delete
7	test	Edit Delete
9	แม่และเด็ก	Edit Delete

- เพิ่มข้อมูล

BSCCT907

เพิ่มข้อมูลหมวดหมู่

ชื่อหมวดหมู่

อนิเมะ

เพิ่มหมวดหมู่

Category

+ เพิ่มข้อมูลหมวดหมู่

Category ID	Category Name	Actions
1	นิยาย	Edit Delete
2	การศึกษา	Edit Delete
3	คอมพิวเตอร์	Edit Delete
4	บ้านและสวน	Edit Delete
5	การ์ตูน	Edit Delete
7	test	Edit Delete
9	แบบด	Edit Delete
12	อนิเมะ	Edit Delete

แก้ไขหมวดหมู่

ชื่อหมวดหมู่

กังฟู

แก้ไขหมวดหมู่

Category

+ เพิ่มข้อมูลหมวดหมู่

Category ID	Category Name	Actions
1	นิยาย	Edit Delete
2	การศึกษา	Edit Delete
3	คอมพิวเตอร์	Edit Delete
4	บ้านและสวน	Edit Delete
5	การ์ตูน	Edit Delete
7	กังฟู	Edit Delete
9	แมมด	Edit Delete
12	อนิเมะ	Edit Delete